



Rheinische
Hochschule
Köln

Fachbereich Ingenieurwesen

Studiengang Informatik (B.Sc.)

Praxistransferprojekt

Aufbau und Evaluation einer GitOps-basierten
Kubernetes-Umgebung auf AWS mit k3s durch
kontrollierte Fehlerexperimente

vorgelegt von
Konstantin Muschallik
Matrikelnr. 653242001

Wintersemester 2026/2027



Rheinische
Hochschule
Köln

Fachbereich Ingenieurwesen

Studiengang Informatik (B.Sc.)

Praxistransferprojekt

Aufbau und Evaluation einer GitOps-basierten
Kubernetes-Umgebung auf AWS mit k3s durch
kontrollierte Fehlerexperimente

vorgelegt von

Konstantin Muschallik

Matrikelnr. 653242001

Erstprüfer: Prof. Dr. Johannes Mauer

Wintersemester 2026/2027

Inhaltsverzeichnis

1. Einleitung	1
1.1. Ausgangssituation und Motivation	1
1.2. Problemstellung und Zielsetzung	1
1.3. Abgrenzung	1
1.4. Leitfrage und Erwartungen	1
1.5. Vorgehensweise und Aufbau der Arbeit	2
2. Praktischer Kontext und Anforderungen	3
2.1. Betrieblicher Hintergrund	3
2.2. Anforderungen an die Betriebsumgebung	3
2.3. Anforderungen an den Praxistransfer	3
3. Technische Grundlagen	4
3.1. Container und Kubernetes	4
3.2. k3s als leichtgewichtige Kubernetes-Distribution	4
3.3. Infrastructure as Code mit Terraform	4
3.4. GitOps und Argo CD	4
3.5. Fehlertoleranz und Selbstheilung in Kubernetes	4
4. Architektur und Entwurfsentscheidungen	6
4.1. Systemarchitektur im Überblick	6
4.2. Wahl von k3s gegenüber Amazon EKS	6
4.3. Zuständigkeitsabgrenzung Terraform und Argo CD	6
4.4. Kustomize zur Manifestverwaltung	6
4.5. Lokale Entwicklungsumgebung und Cloud-Umgebung	7
5. Implementierung	8
5.1. Demo-Anwendung (Resilience Lab)	8
5.2. Terraform-Konfiguration und k3s-Bootstrap	8
5.3. Kubernetes-Manifeste mit Kustomize	8
5.4. Argo-CD-Anbindung und GitOps-Workflow	9
5.5. CI-Pipeline mit GitHub Actions	9
6. Versuchsaufbau und Evaluationsmethodik	10
6.1. Versuchsplan und Messkonfiguration	10
6.2. Basiskonfiguration und verbesserte Konfiguration	10
6.3. Lastprofil und Messwerkzeuge	10
6.4. Definition von Ausfall und Wiederherstellung	10
7. Ergebnisse	11
7.1. Experiment 1: Prozessabsturz und Redundanzvergleich	11

7.2. Experiment 2: Fehlerhaftes Deployment und Git-Revert	11
7.3. Experiment 3: Konfigurationsabweichung und Self-Heal	12
7.4. Zusammenfassung der Gesamtergebnisse	12
8. Diskussion und Praxistransfer	13
8.1. Bewertung der Ergebnisse	13
8.2. Beantwortung der Leitfrage	13
8.3. Einschränkungen und Grenzen	13
8.4. Praxistransfer und betrieblicher Nutzen	13
9. Fazit und Ausblick	14
9.1. Fazit	14
9.2. Ausblick	14
Abkürzungsverzeichnis	15
Abbildungsverzeichnis	16
Tabellenverzeichnis	17
Promptverzeichnis	18
Literaturverzeichnis	19
Verzeichnis der Anhänge	19
Anhang A: Praktische Hinweise zum Gendern	20
Anhang B: Eigenständigkeitserklärung	22
Anhang C: Ergänzende Hinweise für Abschlussarbeiten in den Studiengängen Wirtschaftspsychologie und Mediendesign	23
Anhang D: Gestalterische Hinweise für die Studiengänge Mediendesign (B.A.) und User Experience Design (M.A.)	26
Eigenständigkeitserklärung	27

1. Einleitung

1.1. Ausgangssituation und Motivation

In modernen softwaregetriebenen Unternehmen stellen Verfügbarkeit, Ausfallsicherheit und kurze Releasezyklen von Webanwendungen zentrale Erfolgsfaktoren dar. Cloud-native Architekturen auf Basis von Container-Virtualisierung und Orchestrierungsplattformen wie Kubernetes haben sich hierbei als Industriestandard etabliert. Dennoch bringt der Betrieb komplexer verteilter Systeme signifikante Herausforderungen mit sich: Manuelle Konfigurationsänderungen, Konfigurationsdrift zwischen Entwicklungs- und Produktionsumgebungen sowie unzureichende Resilienzmechanismen führen in der Praxis wiederkehrend zu Dienstunterbrechungen.

Die vorliegende Arbeit untersucht den praktischen Einsatz moderner Betriebsparadigmen, namentlich Infrastructure as Code (IaC) mittels Terraform sowie GitOps unter Verwendung von Argo CD. Als kosteneffiziente und ressourcenschonende Kubernetes-Plattform kommt hierbei k3s auf einer virtuellen Maschine in Amazon Web Services (AWS) zum Einsatz.

1.2. Problemstellung und Zielsetzung

Oftmals wird davon ausgegangen, dass der bloße Einsatz von Kubernetes bereits Hochverfügbarkeit und automatische Selbstreparatur garantiert. In der betrieblichen Realität erfordert eine belastbare Resilienz jedoch ein aufeinander abgestimmtes Zusammenspiel aus Anwendungsarchitektur (z. B. Health Probes), Deklaration des Sollzustands im Versionskontrollsystem und automatisierten Reconcilern.

Ziel dieser Arbeit ist der Aufbau einer vollautomatisierten, reproduzierbaren Labor- und Betriebsumgebung sowie die empirische Überprüfung ihrer Resilienzeigenschaften durch gezielte, kontrollierte Störungsexperimente.

1.3. Abgrenzung

Der Schwerpunkt dieser Arbeit liegt auf der Infrastrukturautomatisierung, dem GitOps-Workflow und der methodischen Fehlerevaluation. Die zu betreibende Webanwendung wird bewusst schlank als zustandslose REST-API mit begleitender Benutzerschnittstelle konzipiert, um Seiteneffekte von Datenbank-Transaktionen oder persistentem Speicher zu isolieren. Auf den Einsatz von Managed Kubernetes Services mit hoher monatlicher Grundgebühr (wie Amazon EKS) wird zugunsten einer transparenten, kostengünstigen k3s-Installation auf AWS EC2 verzichtet.

1.4. Leitfrage und Erwartungen

Aus der Problemstellung leitet sich die zentrale Leitfrage der Arbeit ab:

Inwieweit verbessern automatisierte Bereitstellung mittels Infrastructure as Code und GitOps-

gesteuerte Synchronisation die Ausfallsicherheit und Wiederherstellungsfähigkeit einer betrieblichen Webanwendung auf einer kosteneffizienten Kubernetes-Umgebung gegenüber manuell verwalteten Deployments?

Zur Beantwortung werden folgende überprüfbare Erwartungen formuliert:

1. **Erwartung 1 (Prozess-Resilienz):** Bei einem spontanen Absturz des Anwendungsprozesses stellt die Kubernetes-Plattform den Dienst innerhalb kurzer Zeit automatisch wieder her. Bei redundanter Pod-Auslegung treten keine messbaren Dienstausfälle für Endbenutzer auf.
2. **Erwartung 2 (Deployment-Schutz):** Ein fehlerhaftes Release, dessen Readiness-Prüfung fehlschlägt, führt zu keinem Ausfall des Produktivverkehrs, da bestehende Instanzen aktiv bleiben. Die Wiederherstellung über einen Git-Revert erfolgt deterministisch innerhalb definierter Zeitgrenzen.
3. **Erwartung 3 (GitOps-Selbstheilung):** Manuelle Konfigurationsabweichungen (Configuration Drift) im Cluster werden von Argo CD automatisch erkannt und innerhalb des Abgleichintervalls auf den im Git deklarierten Sollzustand zurückgeführt.

1.5. Vorgehensweise und Aufbau der Arbeit

Die Arbeit gliedert sich in neun Kapitel. Nach der Einführung werden in Kapitel 2 der praktische Kontext und die Anforderungen definiert. Kapitel 3 vermittelt die erforderlichen technischen Grundlagen. In Kapitel 4 wird die Gesamtarchitektur entworfen und begründet. Die praktische Implementierung wird in Kapitel 5 beschrieben. Kapitel 6 legt den Versuchsplan dar, dessen Resultate in Kapitel 7 dokumentiert werden. Kapitel 8 diskutiert die Ergebnisse und den Praxistransfer. Kapitel 9 schließt mit einem Fazit und Ausblick ab.

2. Praktischer Kontext und Anforderungen

2.1. Betrieblicher Hintergrund

In betrieblichen IT-Landschaften nehmen Webservices und Microservices eine zentrale Rolle ein. Ein typisches Szenario stellt die automatisierte Auftrags- und Bestellverarbeitung (Order Processing API) dar, bei der Clients Aufträge einreichen, Statusabfragen durchführen und asynchrone Folgeprozesse angestoßen werden. Dienstunterbrechungen führen in solchen Systemen unmittelbar zu wirtschaftlichen Einbußen und Reputationsverlust.

2.2. Anforderungen an die Betriebsumgebung

Aus dem betrieblichen Szenario leiten sich spezifische funktionale und nicht-funktionale Anforderungen an die Plattform ab:

- **Reproduzierbarkeit:** Die gesamte Infrastruktur muss als Code versioniert und mit minimalem manuellem Eingriff reproduzierbar aufgebaut und zerstört werden können.
- **Transparenz des Systemzustands:** Aktuelle Versionen, Pod-Identitäten und Antwortzeiten müssen für Monitoring und Fehlersuche unmittelbar ersichtlich sein.
- **Fehlertoleranz:** Einzelne Instanzausfälle dürfen nicht zur Nicht-Erreichbarkeit des Gesamtsystems führen.
- **Kosteneffizienz:** Der Betrieb muss innerhalb eines strikten Budgets (unter 30 EUR pro Monat) realisierbar sein.

2.3. Anforderungen an den Praxistransfer

Der Praxistransfer erfordert, dass die im Rahmen dieser Arbeit gewonnenen Erkenntnisse und Implementierungsmuster generalisierbar und direkt auf vergleichbare betriebliche Anwendungen übertragbar sind. Die gewählten Mechanismen (Health Checks, GitOps-Sync, Terraform-Module) dürfen keine proprietären Abhängigkeiten erzeugen, die einen Transfer in reguläre Unternehmensumgebungen verhindern.

3. Technische Grundlagen

3.1. Container und Kubernetes

Containerisierung ermöglicht die Kapselung einer Anwendung inklusive ihrer Laufzeitumgebung und Abhängigkeiten. Kubernetes fungiert als deklarative Orchestrierungsplattform zur Automatisierung von Bereitstellung, Skalierung und Betrieb containerisierter Workloads. Zentrale Konstrukte sind *Pods* als kleinste ausführbare Einheiten, *Deployments* zur Verwaltung des Lebenszyklus und von Rollouts sowie *Services* zur internen und externen Lastverteilung.

3.2. k3s als leichtgewichtige Kubernetes-Distribution

k3s ist eine von der Cloud Native Computing Foundation (CNCF) zertifizierte, leichtgewichtige Kubernetes-Distribution. Durch die Bündelung aller Kubernetes-Komponenten in einer einzelnen Binärdatei und den Ersatz des speicherintensiven etcd durch SQLite (bei Einzel-Node-Installationen) wird der Arbeitsspeicherbedarf der Control Plane signifikant reduziert. Damit eignet sich k3s ideal für ressourcenschonende Cloud-Installationen und Edge-Szenarien, ohne Kompatibilitätseinbußen gegenüber Standard-Kubernetes aufzuweisen.

3.3. Infrastructure as Code mit Terraform

Infrastructure as Code (IaC) beschreibt die deklarative Definition von IT-Infrastruktur in maschinenlesbaren Konfigurationsdateien. HashiCorp Terraform nutzt die HashiCorp Configuration Language (HCL) und führt Zustandsabgleiche über State-Dateien durch. Dadurch werden Änderungen nachvollziehbar, versionierbar und idempotent ausführbar.

3.4. GitOps und Argo CD

GitOps erweitert das IaC-Paradigma auf den Betrieb von Kubernetes-Workloads: Ein Git-Repository dient als Single Source of Truth für den gesamten gewünschten Clusterzustand (Desired State). Ein im Cluster laufender Agent – wie Argo CD – überwacht kontinuierlich Abweichungen zwischen dem Soll-Zustand in Git und dem Ist-Zustand im Cluster und synchronisiert diese automatisch oder manuell.

3.5. Fehlertoleranz und Selbstheilung in Kubernetes

Die Selbstreparatur in Kubernetes stützt sich auf Controller-Schleifen (Reconciliation Loops) und definierte Health Checks:

- **Restart Policy:** Bestimmt das Verhalten des Kubelets beim Beenden von Containern.
- **Liveness Probes:** Erkennen Deadlocks und interne Hänger; ein Fehlschlag führt zum Containerneustart.
- **Readiness Probes:** Steuern, ob ein Pod bereit ist, Anfragen über den zugeordneten

Service entgegenszunehmen; ein Fehlschlag entzieht den Pod vorübergehend dem Load Balancing.

4. Architektur und Entwurfsentscheidungen

4.1. Systemarchitektur im Überblick

Die Gesamtlösung gliedert sich in drei funktionale Schichten:

1. **Infrastrukturschicht (AWS):** Eine Virtuelle Private Cloud (VPC) mit öffentlichen Subnetzen, Internet Gateway, Security Groups sowie einer EC2-Instanz ('t3.small') mit fest zugewiesener Elastic IP.
2. **Plattformschicht (k3s & Argo CD):** k3s stellt die containerbasierte Laufzeitumgebung bereit. Argo CD überwacht als GitOps-Controller das Anwendungs-Repository.
3. **Anwendungsschicht (Resilience Lab):** Eine zustandslose Webanwendung, bestehend aus REST-API, Statusoberfläche und geschütztem Fehlerinduktionsmodul, strukturiert über Kustomize.

4.2. Wahl von k3s gegenüber Amazon EKS

Ein zentraler Entwurfsaspekt betrifft das Verhältnis von Betriebskosten und Funktionalität. Während Amazon EKS eine verwaltete Control Plane bietet, entstehen hierfür fixe Basiskosten von ca. 72 USD pro Monat allein für den Cluster-Endpunkt, zuzüglich Worker Nodes und Load Balancern (Gesamtkosten ca. 180 USD/Monat).

k3s auf einer 't3.small'-Instanz bietet bei monatlichen Kosten von ca. 22 USD die volle Kubernetes-API-Kompatibilität. Da die geplanten Resilienzuntersuchungen auf Pod- und Deployment-Ebene stattfinden, bietet k3s dieselbe Aussagekraft bei einem Bruchteil der Kosten.

4.3. Zuständigkeitsabgrenzung Terraform und Argo CD

Um Konflikte durch konkurrierende Controller zu vermeiden, gilt ein striktes Separationsprinzip:

- **Terraform** verwaltet ausschließlich Cloud-Ressourcen (VPC, EC2, Security Groups) und initiiert den einmaligen k3s- und Argo-CD-Bootstrap via Cloud-Init.
- **Argo CD** verwaltet alle Kubernetes-Anwendungsressourcen (Deployments, Services, Ingress). Terraform greift nach dem Bootstrap nicht mehr modifizierend in Anwendungsmanifeste ein.

4.4. Kustomize zur Manifestverwaltung

Die Kubernetes-Konfiguration wird mittels Kustomize verwaltet. Im Vergleich zu Helm verzichtet Kustomize auf eine eigene Template-Sprache und arbeitet deklarativ mit Plain-YAML-Manifesten. Über ein Base-Overlay-Prinzip werden gemeinsame Ressourcen in

einer Basis (`base`) definiert und umgebungsspezifische Anpassungen (`overlays/local` für lokale Cluster, `overlays/aws` für k3s mit Traefik-Ingress) per Patch überlagert. Dies reduziert Redundanz, vermeidet Template-Fehler und ist nativ in `kubectl` sowie Argo CD integriert.

4.5. Lokale Entwicklungsumgebung und Cloud-Umgebung

Um Cloud-Kosten zu minimieren, wird der Entwicklungszyklus lokal vorangetrieben (Containerisierung mit Docker, lokale Validierung). AWS dient gezielt der Durchführung und Validierung der definierten Experimente unter realistischen Netzwerkbedingungen.

5. Implementierung

5.1. Demo-Anwendung (Resilience Lab)

Die Webanwendung wird auf Basis von Python (FastAPI) implementiert. Sie stellt Kernendpunkte für den regulären Betrieb sowie dedizierte Schnittstellen zur Fehlerinduktion bereit:

- `GET /`: Übersichtliche HTML-Statusseite mit Anzeige von Pod-Name, Anwendungs-version, Node-IP und Systemzeit.
- `GET /health/live`: Liveness-Prüfung zur Erkennung interner Deadlocks.
- `GET /health/ready`: Readiness-Prüfung zur Steuerung der Service-Teilnahme.
- `POST /lab/crash`: Forcierter Programmabsturz via `os._exit(1)`.
- `POST /lab/toggle-ready`: Umschaltung der Bereitschaft für kontrollierte Traffic-Aussteuerungen.

5.2. Terraform-Konfiguration und k3s-Bootstrap

Die AWS-Infrastruktur wird über modulare Terraform-Dateien beschrieben:

- `vpc.tf`: Definiert VPC, Subnetze, Internet Gateway und Routing-Tabellen.
- `security-groups.tf`: Konfiguriert Inbound-Regeln für SSH (Port 22), HTTP/HTTPS (Port 80/443) und die Kubernetes-API (Port 6443).
- `ec2.tf`: Instanziiert die VM unter Verwendung eines `cloud-init`-Skripts, welches k3s und Argo CD installiert und das Kubeconfig-Zertifikat mit der öffentlichen IP ausstellt.

5.3. Kubernetes-Manifeste mit Kustomize

Die Anwendungsbereitstellung wird über eine strukturierte Kustomize-Hierarchie realisiert:

- `kustomize/base`: Beinhaltet Basis-Manifeste für Deployment und Service mit Standardwerten (1 Replikat, Ressourcenlimits, Health Probes).
- `kustomize/overlays/local`: Patcht den Service auf `NodePort 30080` und nutzt lokale Container-Images für Entwicklungszwecke.
- `kustomize/overlays/aws`: Skaliert das Deployment auf 2 Replikate für Resilienztests und fügt ein Ingress-Manifest für den k3s-integrierten Traefik-Controller hinzu.

5.4. Argo-CD-Anbindung und GitOps-Workflow

Die Bereitstellung der Anwendung erfolgt über eine Argo-CD-Application-CRD. Argo CD überwacht das Git-Repository auf dem Pfad `kustomize/overlays/aws` und synchronisiert den Zustand automatisiert (*Automated Sync & Self-Heal*).

5.5. CI-Pipeline mit GitHub Actions

Eine GitHub-Actions-Pipeline automatisiert den Integrationsprozess: Bei Änderungen am Anwendungscode werden automatisierte Tests ausgeführt, ein Container-Image gebaut und in die GitHub Container Registry (`ghcr.io`) übertragen.

6. Versuchsaufbau und Evaluationsmethodik

6.1. Versuchsplan und Messkonfiguration

Um reproduzierbare Ergebnisse zu erzielen, folgt jeder Störungsversuch einem standardisierten Ablauf:

1. Herstellung eines definierten Ausgangszustands (Baseline) mit bekannter Commit-ID und Image-Version.
2. Start der kontinuierlichen Hintergrundlast (k6-Lasttest).
3. Gezielte Induktion der Störung zum definierten Zeitpunkt t_{fault} .
4. Kontinuierliche Aufzeichnung von Fehlerraten und Antwortzeiten bis zur vollständigen Stabilisierung $t_{\text{recovered}}$.
5. Protokollierung und Export der Rohdaten.

6.2. Basiskonfiguration und verbesserte Konfiguration

Für die Experimente werden jeweils zwei Szenarien verglichen:

- **Basiskonfiguration (Single Replica):** Minimalauslegung mit exakt einer Pod-Instanz ohne PodDisruptionBudget.
- **Verbesserte Konfiguration (Multi Replica):** Redundante Auslegung mit mindestens zwei Pod-Instanzen, feingranularen Health Probes und Rolling-Update-Strategie.

6.3. Lastprofil und Messwerkzeuge

Zur Lastgenerierung wird das Open-Source-Tool k6 eingesetzt. Ein konstanter Anfragestrom von 10 bis 20 virtuellen Benutzern (VUs) fragt den Statusendpunkt ab. Gemessen werden HTTP-Statuscodes, Fehlerraten sowie die Latenzperzentile (Median p50 und 95. Perzentil p95).

6.4. Definition von Ausfall und Wiederherstellung

Ein Systemzustand gilt als **ausgefallen**, sobald die HTTP-Erfolgsrate (Statuscode 200) unter 99 % fällt oder Timeouts auftreten. Die **Wiederherstellungszeit** (Mean Time to Recovery, MTTR) bezeichnet die Zeitspanne vom Auftreten des ersten Fehlers bis zum Erreichen eines stabilen Betriebs ohne Fehlantworten über ein Intervall von mindestens 30 Sekunden.

7. Ergebnisse

7.1. Experiment 1: Prozessabsturz und Redundanzvergleich

In Experiment 1 wurde die Auswirkung eines plötzlichen Prozessabsturzes unter kontinuierlicher Hintergrundlast (15 Anfragen pro Sekunde über HTTP) untersucht. Verglichen wurde eine Basiskonfiguration mit einem einzelnen Pod gegen eine verbesserte Konfiguration mit zwei redundanten Pods. Bei Sekunde $t = 12$ s wurde ein abruptes Beenden des Containerprozesses mittels `os._exit(1)` über den Labor-Endpunkt forciert.

Konfiguration	Anfragen	Erfolg (200)	Fehler	Fehlerrate	p_{50} Latenz	p_{95} Latenz
Basiskonfiguration (1 Pod)	525	517	8	1,52 %	50,75 ms	162,06 ms
Verbesserte Konfig. (2 Pods)	525	518	7	1,33 %	55,20 ms	159,81 ms

Tab. 1: Messergebnisse zu Experiment 1 (Prozessabsturz unter Last), eigene Messung.

Das Kubelet erkannte den abgestürzten Prozess in beiden Fällen und startete den Container binnen ein bis zwei Sekunden automatisch neu. Die verbleibende Fehlerrate von ca. 1,3 % bis 1,5 % resultiert aus den Verbindungen, die sich exakt zum Absturzzeitpunkt im TCP-Handshake befanden. Bei zwei Pods stand dem Gesamtsystem jedoch unterbrechungsfrei die verbleibende Instanz für neu eintreffende Anfragen zur Verfügung.

7.2. Experiment 2: Fehlerhaftes Deployment und Git-Revert

Experiment 2 überprüfte den Schutzmechanismus der Rolling-Update-Strategie in Kombination mit Readiness-Probes sowie die Wiederherstellungsdauer bei einem fehlerhaften Release. Über Commit 05e2318 wurde eine fehlerhafte Version ausgerollt, deren Readiness-Prüfung fehlschlug.

Phase	Dauer	Anfragen	Fehlerrate	Verhalten des Clusters
Fehlerhafter Rollout	20,0 s	200	0,0 %	Neuer Pod 0/1 NotReady, Rollout pausiert
Git-Revert & Sync	37,2 s	—	—	Argo CD rollt alten Zustand zurück, Pod terminiert

Tab. 2: Messergebnisse zu Experiment 2 (Fehlerhaftes Deployment & Revert), eigene Messung.

Da der neu erzeugte Pod die Readiness-Bedingung nicht erfüllte, wurde er zu keinem Zeitpunkt in die Service-Endpoints aufgenommen. Während des gesamten Fehlversuchs blieb der Produktivverkehr zu 100 % fehlerfrei (**0,0 % Fehlerrate**). Nach Ausführung eines `git revert` synchronisierte Argo CD den vorherigen Zustand autonom innerhalb von **37,19 Sekunden**.

7.3. Experiment 3: Konfigurationsabweichung und Self-Heal

Dieses Experiment untersuchte die Selbstreparaturfähigkeit der GitOps-Pipeline bei manuellen Eingriffen im Cluster. Das Deployment `resilience-lab` wurde manuell über `kubectl scale --replicas=0` stillgelegt.

Zeitpunkt	Argo CD Sync	Ist-Replikate	Ereignis
$t_0 = 0,0 \text{ s}$	Synced	2	Ausgangszustand vor Störung
$t_1 = 0,1 \text{ s}$	Synced	0	Manuelle Skalierung auf 0 abgesetzt
$t_2 = 2,6 \text{ s}$	OutOfSync	0	Argo CD detektiert Drift ($t_{\text{detect}} = 2,58 \text{ s}$)
$t_3 = 4,7 \text{ s}$	Synced	0	Self-Heal initiiert Wiederherstellung
$t_4 = 11,3 \text{ s}$	Synced	2	Beide Pods gestartet und 1/1 Ready

Tab. 3: Messergebnisse zu Experiment 3 (Configuration Drift und Self-Heal), eigene Messung.

Argo CD detektierte die Konfigurationsabweichung innerhalb von $t_{\text{detect}} = 2,58 \text{ s}$. Die vollständige Wiederherstellungszeit bis zum Erreichen von zwei gesunden Replikaten betrug $\text{MTTR} = 11,28 \text{ s}$.

7.4. Zusammenfassung der Gesamtergebnisse

Alle drei Kernexperimente belegen empirisch die Hypothesen der Arbeit: GitOps eliminiert Konfigurationsdrift autonom binnen Sekunden, Readiness-Checks verhindern Ausfälle bei Fehl-Releases vollständig, und Kubernetes-Replikate sichern die Grundverfügbarkeit bei unvorhergesehenen Prozessabstürzen ab.

8. Diskussion und Praxistransfer

8.1. Bewertung der Ergebnisse

Die Experimente demonstrieren, dass Containerorchestrierung in Verbindung mit GitOps eine signifikante Erhöhung der Betriebssicherheit bietet. Gleichzeitig wird deutlich, dass Resilienz nicht allein durch die Plattform erzeugt wird: Nur korrekt konfigurierte Readiness- und Liveness-Prüfungen verhindern, dass fehlerhafter Code ungefiltert Endnutzer erreicht.

8.2. Beantwortung der Leitfrage

Die eingangs gestellte Leitfrage kann positiv beantwortet werden: Die Kombination aus deklarativer Infrastruktur mit Terraform und kontinuierlichem GitOps-Abgleich mit Argo CD eliminiert manuelle Betriebsrisiken, verkürzt die Mean Time to Recovery (MTTR) drastisch und stellt eine reproduzierbare Wiederherstellung nach Störungen sicher.

8.3. Einschränkungen und Grenzen

Als wesentliche Limitation ist die Beschränkung auf einen Single-Node-Cluster im Rahmen dieser Arbeit zu nennen. Ein physischer Ausfall der gesamten EC2-Instanz kann von internen Kubernetes-Mechanismen nicht kompensiert werden; hierfür wäre ein Multi-Node-Cluster über mehrere Verfügbarkeitszonen (Availability Zones) erforderlich, was mit höheren Betriebskosten verbunden ist.

8.4. Praxistransfer und betrieblicher Nutzen

Für betriebliche Anwendungen liefert das vorgestellte Modell eine kosteneffiziente Blaupause: Entwicklungs- und Stagingumgebungen können mit minimalem Budget auf k3s betrieben werden, während die gleichen Helm-Charts und GitOps-Pipelines ohne Anpassung auf Multi-Node-EKS-Produktionsclustern eingesetzt werden können.

9. Fazit und Ausblick

9.1. Fazit

Die vorliegende Arbeit hat gezeigt, dass moderne DevOps- und GitOps-Verfahren auch mit limitierten Cloud-Budgets praxisnah evaluiert werden können. Die Implementierung auf Basis von k3s auf AWS EC2 in Kombination mit Terraform und Argo CD erwies sich als hochgradig stabil und reproduzierbar. Die kontrollierten Störungsexperimente lieferten klare, quantifizierbare Nachweise für die Resilienzgewinne durch automatisierte Selbstreparatur und deklaratives Konfigurationsmanagement.

9.2. Ausblick

Künftige Arbeiten könnten die Umgebung um Multi-Cluster-Setups erweitern, Chaos-Engineering-Werkzeuge wie Chaos Mesh für stochastische Ausfallsimulationen integrieren oder das Verhalten zustandsbehafteter Datenbank-Workloads unter Störungseinfluss untersuchen.

Abkürzungsverzeichnis

API – Application Programming Interface

AWS – Amazon Web Services

CD – Continuous Delivery / Continuous Deployment

CI – Continuous Integration

CNCF – Cloud Native Computing Foundation

CPU – Central Processing Unit

CRD – Custom Resource Definition

EC2 – Elastic Compute Cloud

EIP – Elastic IP Address

GHCR – GitHub Container Registry

HCL – HashiCorp Configuration Language

HTTP – Hypertext Transfer Protocol

IaC – Infrastructure as Code

JSON – JavaScript Object Notation

K8s – Kubernetes

MTTR – Mean Time to Recovery

RAM – Random Access Memory

REST – Representational State Transfer

SSH – Secure Shell

URI – Uniform Resource Identifier

VPC – Virtual Private Cloud

Abbildungsverzeichnis

Tabellenverzeichnis

Tab. 1	Messergebnisse zu Experiment 1 (Prozessabsturz unter Last), eigene Messung.	11
Tab. 2	Messergebnisse zu Experiment 2 (Fehlerhaftes Deployment & Revert), eigene Messung.	11
Tab. 3	Messergebnisse zu Experiment 3 (Configuration Drift und Self-Heal), eigene Messung.	12

Promptverzeichnis

Verzeichnis der Anhänge

Anhang A:	Praktische Hinweise zum Gendern	20
Anhang B:	Eigenständigkeitserklärung	22
Anhang C:	Ergänzende Hinweise für Abschlussarbeiten für Abschlussarbeiten in den Studiengängen Wirtschaftspsychologie und Mediendesign	23
Anhang D:	Gestalterische Hinweise für die Studiengänge Mediendesign (B.A.) und User Experience Design (M.A.)	26

Anhang A: Praktische Hinweise zum Gendern

Eine häufig verwendete Möglichkeit ist das substantivierte Partizip Präsens. Es zeigt im Plural kein Genus:

- die Studierenden,
- die Lehrenden.

Bitte beachten Sie, dass das substantivierte Partizip Präsens im Singular ein Genus zeigt:

- der Studierende ist eindeutig maskulin,
- die Lehrende ist eindeutig feminin.

Abstrakte Rollenbezeichnungen und Begriffe für Kollektive, Institutionen und Positionen sind eine weitere Möglichkeit für sexusindifferente Formulierungen:

- die Vertretung,
- die Studiengangsleitung,
- das Kollegium,
- das Publikum,
- die Lehrkräfte,
- das Team,
- das Präsidium.

Die Pronomina alle und niemand zeigen ebenfalls kein Genus:

- Alle können am Kurs teilnehmen.
- Niemand muss sich wegen der Prüfung Sorgen machen.

Gender-Zeichen, meist ein Doppelpunkt oder Sternchen (Asterisk), stehen zwischen dem Wortstamm der männlichen Form und der weiblichen Endung:

- Bewerber:in
- Teilnehmer:innen
- Student:in
- Expert:innen

Die Beidnennung ist eine weitere Möglichkeit, also die Nennung von maskulinen und femininen Bezeichnungen:

- Studentinnen und Studenten,
- Dozentinnen und Dozenten,
- Prüferin bzw. Prüfer,
- Expertin bzw. Experte.

Anhang B: Eigenständigkeitserklärung

Ihre Eigenständigkeitserklärung hat folgenden Wortlaut (die englische Version verwenden Sie nur, wenn Sie Ihre Arbeit auf Englisch verfassen). Bitte denken Sie daran, sie eigenhändig zu unterschreiben.

Eigenständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen Publikationen, Vorlagen und Hilfsmittel (z.B. künstliche Intelligenz) als die angegebenen benutzt habe. Alle Teile meiner Arbeit, die wortwörtlich oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Gleiches gilt für von mir verwendete Internetquellen. Ich versichere, dass ich diese Arbeit oder nicht zitierte Teile daraus vorher nicht in einem anderen Prüfungsverfahren eingereicht habe. Mir ist bekannt, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs mittels einer Plagiatserkennungssoftware auf eine ungekennzeichnete Übernahme von fremdem geistigen Eigentum sowie auf die Nutzung von künstlicher Intelligenz zur Texterstellung überprüft werden kann. Ich versichere, dass die elektronische Form meiner Arbeit mit der gedruckten Version identisch ist.

Datum, Unterschrift

Declaration of Originality

I hereby confirm that I have independently written this work and have not used any publications, templates, or aids (e.g. artificial intelligence) other than those I have indicated. All parts of my work which have been taken literally or correspondingly from other publications have been duly acknowledged. This also applies to Internet sources. I confirm that I have not previously submitted this work or any unquoted parts thereof in any other examination procedure. I am aware that my work may be checked for plagiarism by means of plagiarism recognition software, as well as for the use of artificial intelligence for text creation, in order to verify the integrity of its written content. I also confirm that the electronic form is identical to the printed version.

Date, signature

Anhang C: Ergänzende Hinweise für Abschlussarbeiten in den Studiengängen Wirtschaftspsychologie und Mediendesign

In diesen beiden Studiengängen können die Abschlussarbeit unterschiedlich ausgerichtet werden. Möglich sind:

1. theoretisch-konzeptionelle Abschlussarbeiten,
2. empirische Arbeiten (in der Wirtschaftspsychologie verpflichtend): qualitative oder quantitative Untersuchungen,
3. konzeptionell-gestalterische Arbeiten (nur im Studiengang Mediendesign).

Eine eindeutige Zuordnung zu einem dieser Schwerpunkte ist nicht immer möglich.

In jedem Fall ist eine umfassende und systematische Darstellung des Stands der Forschung anhand der aktuellen wissenschaftlichen Literatur (Diskurs) erforderlich. Auf diesen Status Quo bezieht sich Ihre weitere Bearbeitung, bzw. Sie wenden ihn dafür an.

Alle Arbeiten können in Kooperation mit Unternehmen geschrieben werden. Bitte stimmen Sie sich dazu mit Ihren Prüfern ab.

1. Theoretisch-konzeptionelle Arbeiten

Wenn Sie eine theoretisch-konzeptionelle Arbeit vorlegen, analysieren Sie darin ein Thema, ohne dass Sie hierzu eigene Berechnungen durchführen.

Ihr Ausgangspunkt ist eine möglichst vollständige Erfassung der relevanten Literatur und die systematische Aufbereitung der gewonnenen Informationen. Wie Sie dieses Material strukturieren und aufbereiten (Tiefe, Differenzierung, Präzision, Prägnanz, Gewichtung), bestimmt wesentlich die Qualität Ihrer Arbeit.

Typischerweise wird von Ihnen erwartet, dass Sie die wesentliche Literatur durchdringen und sich mit ihr kritisch auseinandersetzen. Dazu zählen insbesondere Definitionen (die Sie evtl. selbst entwickeln), Klassifikationsschemata oder Typologien. Zudem wird erwartet, dass sie Begründungszusammenhänge strukturierte darstellen.

Ein zentrales Ziel der theoretisch-konzeptionellen Arbeit ist die Verbesserung, Vereinfachung und vor allem Systematisierung der Darstellung bereits bekannter, aber bisher nicht in einer einzigen Arbeit zusammengefasster Fakten oder Argumente.

Den Titel Ihrer theoretisch-konzeptionellen Arbeit können Sie beispielsweise nach diesem Schema formulieren:

- Entwicklung einer strategisch-konzeptionellen Empfehlung
- Formulierung von Handlungsempfehlungen für bestimmte Fälle

- Eingrenzung von potenziellen Entwicklungsszenarien
- Konzeption von bestimmten Maßnahmenempfehlungen für besondere Zwecke (beispielsweise aus dem konkreten Arbeitsumfeld).

Ihre Strukturierungsleistung soll zu einem Ziel führen, das bereits im Titel anklingt, z.B.: „Strategische Markt- und Wettbewerbsanalyse der Videospielbranche - unter besonderer Berücksichtigung der Neuprodukteinführung eines Konsolenherstellers“.

2. Empirische Arbeiten

Bei Ihrer empirischen Arbeit kann es sich entweder um eine qualitative oder quantitative Untersuchung handeln.

Qualitative Untersuchungen werden durchgeführt, wenn ein wesentliches Verständnis für eine bestimmte Forschungsfrage erlangt werden soll. Ziel der qualitativen Forschung ist das Erkennen, Beschreiben und Verstehen von Zusammenhängen. Im Vordergrund stehen die vollständige Erfassung und Interpretation aller problemrelevanten Aspekte hinsichtlich des Themas der Abschlussarbeit. Hierbei bedient man sich offener, nicht standardisierter Erhebungsverfahren (z.B. Interviews, Expertengespräche, Gruppendiskussionen, qualitative Beobachtung, qualitative Experimente), deskriptiver Aufbereitungsverfahren (z.B. Gesprächsprotokoll, Transkription) und interpretativer Auswertungsverfahren (z.B. qualitative Inhaltsanalyse, Cognitive Mapping, objektive Hermeneutik). In der Regel kommen im Rahmen der qualitativen Untersuchung Methoden zum Einsatz, die sich auf kleine Fallzahlen beschränken, keine statistischen Analysen (z.B. Signifikanztests) implizieren, relative weiche Daten produzieren und ihre Erkenntnisse auf einem verhältnismäßig niedrigen Abstraktionsniveau mittels subjektiver Interpretation gewinnen. Die gewonnenen Ergebnisse sind zwar nicht repräsentativ, sie dienen aber zur ausführlichen Stoffsammlung, um ihrerseits wieder Hypothesen quantifizieren zu können.

Quantitative Untersuchungen haben die Messung bestimmter Sachverhalte bzw. die Entdeckung von Gesetzmäßigkeiten zum Gegenstand. Es handelt sich hierbei um einen Ansatz, der theoriegeleitet ist und der sich standardisierter Erhebungsmethoden (i.d.R. schriftliche Fragebögen) bedient. Ziel dieser Standardisierung ist es, die Antworten einer Vielzahl von Befragten unmittelbar vergleichen zu können. Der Vorteil quantitativer Untersuchungen liegt darin, dass sich die Messergebnisse mit statistischen Methoden (z.B. Korrelations- und Regressionsanalysen, Varianzanalysen, Faktoren- und Clusteranalysen, Metaanalysen, Diskriminanzanalysen, MDS) unter Nutzung statistischer Kennzahlen (Mittelwert, Median, Standardabweichung, Varianz etc.) verdichten und weiterverarbeiten lassen. Die Auswertung und Analyse der Daten, die im Rahmen der quantitativen Forschung generiert werden, erfolgt mit Hilfe von statistischen Programmen (z.B. SPSS, AMOS, PLS, R). Ziel ist es, Zusammenhänge zu erkennen und daraus allgemein gültige Aussagen abzuleiten. Rückschlüsse auf die tatsächlichen Verhältnisse in der Grundgesamtheit sind möglich.

In der Wirtschaftspsychologie (B.Sc.) werden ausschließlich empirische Abschlussarbeiten durchgeführt, wobei quantitativen Ansätzen der Vorzug zu geben ist.

3. Konzeptionell-gestalterische Arbeiten

Diese Arbeit können Sie nur im Studiengang Mediendesign vorlegen. Hierbei handelt es sich um eine innovative Lösung für eine komplexe gestalterische Aufgabe.

Ihr Ausgangspunkt ist auch hier die Durchdringung relevanter wissenschaftlicher Grundlagen (insbesondere Theorien und Beispiele aus der Designgeschichte).

Daran schließen sich evtl. strategische sowie Ihre konzeptionellen und methodischen Darstellungen an, in denen Sie den Kontext der Aufgabe und Ihren Designprozess nachvollziehbar machen.

Sowohl die wissenschaftlichen als auch die methodischen und konzeptionellen Aussagen führen zu Ihrer praktischen Umsetzung im Sinn einer Lösungsentwicklung mit gestalterischen und technischen Mitteln.

Anhang D: Gestalterische Hinweise für die Studiengänge Mediendesign (B.A.) und User Experience Design (M.A.)

Schmutztitel und Haupttitel

Auf dem Schmutztitel und dem Haupttitel stehen die gleichen Inhalte wie bei allen anderen Studiengängen (vgl. Abschnitt 6.6 in diesem Leitfaden). Ansonsten sind Sie in Ihrer Gestaltung der Titel frei. Es ist auch ausdrücklich erwünscht, dass Sie von dieser Freiheit Gebrauch machen. Falls Sie sich dafür entscheiden, Ihre Arbeit nicht zu gestalten, verwenden Sie einen braunen (Mediendesign) bzw. weißen (User Experience Design) Einband. Bitte beachten Sie, dass Sie keinen schlichten andersfarbigen Einband verwenden, weil diese bereits anderen Studiengängen vorbehalten sind.

Schriften

Beachten Sie bei der Wahl Ihrer Schrift die Angemessenheit. Neben der akademischen Ausstrahlung ist es wichtig, dass Sie sich später als Bewerber mit Ihrer Arbeit gegen Konkurrenten durchsetzen wollen. Alle Schriften für sogenannte Mengentexte sind aufgrund ihrer Lesefreundlichkeit geeignet, beispielsweise die Thesis, Info Text oder Corporate oder Schriften nach dem dynamischen Formprinzip, wie z.B. Garamond, Minion, Weidemann, Lexicon, Optima, Officina, Frutiger, Today oder Meta.

Beachten Sie bitte auch, dass die Prüfer Ihre Arbeit innerhalb kurzer Zeit lesen werden. Die Lektüre darf auch Freude bereiten, deshalb ist die Lesbarkeit bei der Auswahl ein wichtiges Entscheidungskriterium. Eine schmal laufende Schrift (condensed) ist also nicht geeignet. Serifenschriften bieten in der Regel Vorteile, sind aber kein Muss.

Raster und Layout

Die Bachelor-Thesis soll angemessene Seitenränder haben und nur eine Textspalte, sowohl einen toten als auch einen lebenden Kolumnentitel integrieren sowie zwingend über eine Paginierung verfügen. Abbildungen sollt angemessen groß sein. Komplexe Schaubilder oder die Herleitung Ihrer Entwürfe dürfen, sofern notwendig, seitenfüllend sein. Für Infografiken entwickeln Sie idealerweise einen durchgängigen einheitlichen Stil.

Feintypografie

Beachten Sie bitte alle feintypografischen Regeln. Halten Sie bitte sauber definierte Abstände vor und nach Überschriften oder Absätzen usw. ein. Ein Ausgleich des Flatter- oder Blocksatzes nach Fertigstellen des Textes ist Pflicht. Tipp: Beginnen Sie damit von hinten.

Eigenständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen Publikationen, Vorlagen und Hilfsmittel (z.B. künstliche Intelligenz) als die angegebenen benutzt habe. Alle Teile meiner Arbeit, die wortwörtlich oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Gleiches gilt für von mir verwendete Internetquellen. Ich versichere, dass ich diese Arbeit oder nicht zitierte Teile daraus vorher nicht in einem anderen Prüfungsverfahren eingereicht habe. Mir ist bekannt, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs mittels einer Plagiatserkennungssoftware auf eine ungekennzeichnete Übernahme von fremdem geistigen Eigentum sowie auf die Nutzung von künstlicher Intelligenz zur Texterstellung überprüft werden kann. Ich versichere, dass die elektronische Form meiner Arbeit mit der gedruckten Version identisch ist.

[Ort], 1. Oktober 2026

Ort, Datum

Unterschrift